



МІНІСТЕРСТВО
ОСВІТИ І НАУКИ
УКРАЇНИ



ХАРКІВСЬКИЙ
НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ

Веб-система прогнозування умов занурення на основі аналізу океанографічних та метеорологічних даних

Комплексний курсовий проєкт

Виконала: студентка групи ПЗПІ-23-5

Мельник Анастасія Олександрівна

Керівник: ст. викладач Терещенко Г. Ю.

Харків 2026

Мета та актуальність

Мета роботи

- спроектувати й реалізувати повностековий веб-застосунок iSea, що формує єдину оцінку безпеки занурення для будь-якої точки океану;

Актуальність

- у світі понад 28 млн сертифікованих дайверів, їх кількість зростає;
- безпека занурення залежить від десятків параметрів довкілля;
- наявні застосунки не дають єдиної інтегральної оцінки умов.

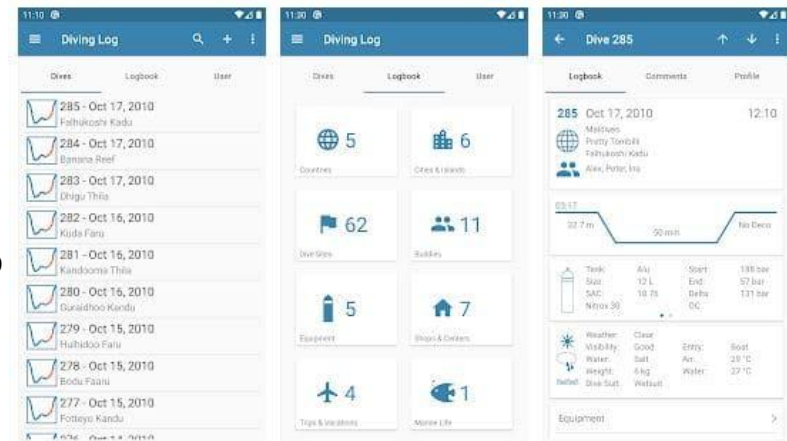
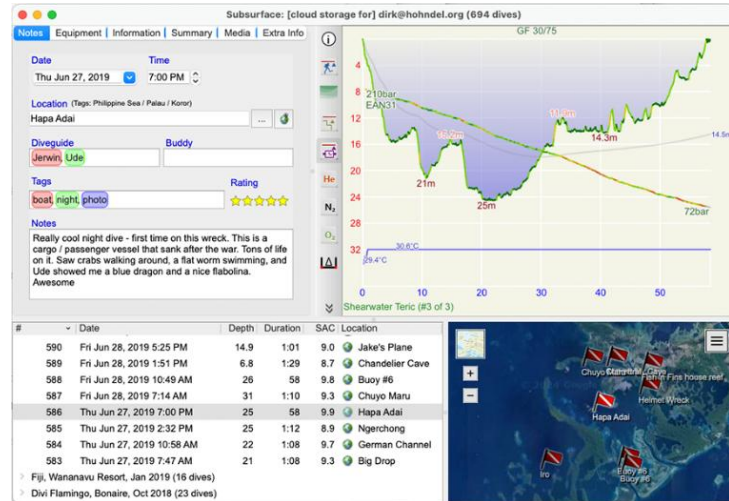
Аналіз існуючих рішень

Досліджені аналоги

- Subsurface — відкритий дайв-лог з підтримкою дайв-комп'ютерів;
- Dive Log — мобільний каталог занурень; Windy.com — метеопортал;
- локальні дайв-портали (каталоги дайв-сайтів).

Спільні прогалини

- немає батиметрії, моделювання термоклина та підводної видимості;
- немає агрегованого рейтингу безпеки й рольового доступу.



Постановка задачі

Проблема: автоматизувати збір і аналіз різнорідних даних та сформувати єдину оцінку безпеки занурення.

Основні задачі

- багаторівнева архітектура на .NET 10, доменна модель у PostgreSQL;
- рольовий доступ: Diver, Trainer, DiveCenter, Admin;
- конвеєр оцінки з 10 етапів та відмовостійка інтеграція (SafeRun);
- імпорт з OpenStreetMap, клієнт на React 19 з інтерактивною мапою;
- автоматизоване тестування та контейнеризоване розгортання.

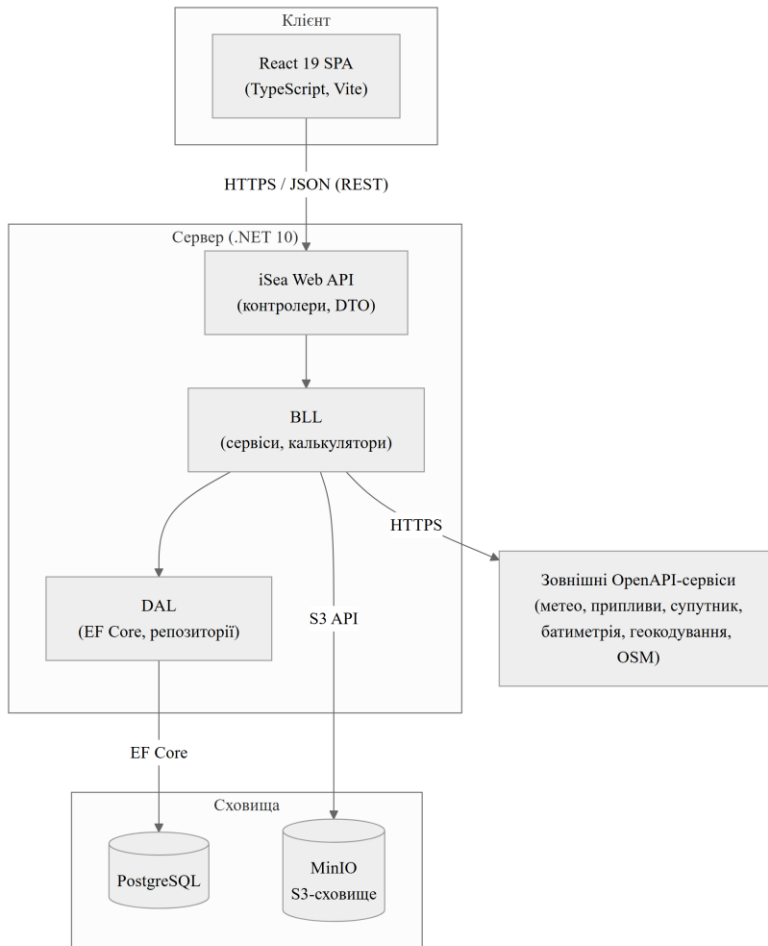
Технології розробки

- **Сервер:** .NET 10, ASP.NET Core, EF Core 10, PostgreSQL, MinIO, JWT.
- **Клієнт:** React 19, TypeScript, Vite, Tailwind CSS 4, React Router 7, Leaflet.
- **Інфраструктура:** Docker, Docker Compose, хмарне розгортання на Railway.
- **Зовнішні дані:** метео- й морські прогнози, припливи, супутниковий аналіз, батиметрія, геокодування, OSM.

Архітектура системи

Clean Architecture, п'ять шарів:

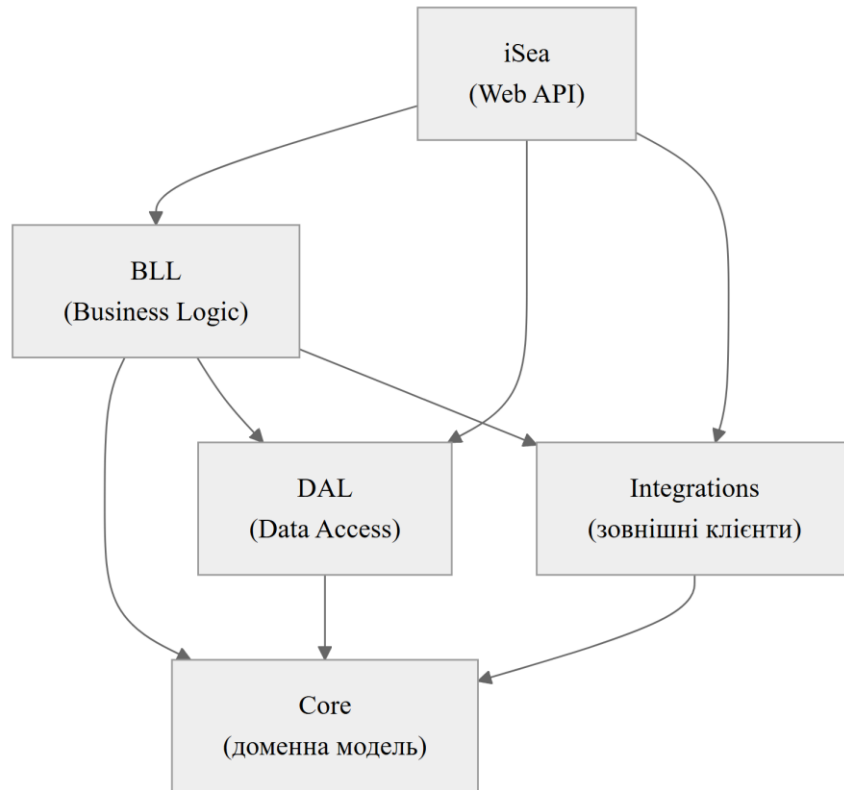
- iSea — Web API (контролери, DTO);
- BLL — бізнес-логіка й калькулятори;
- DAL — доступ до даних (EF Core);
- Core — доменна модель;
- Integrations — зовнішні сервіси.



Компоненти та проєкту

Шість проєктів у складі solution:

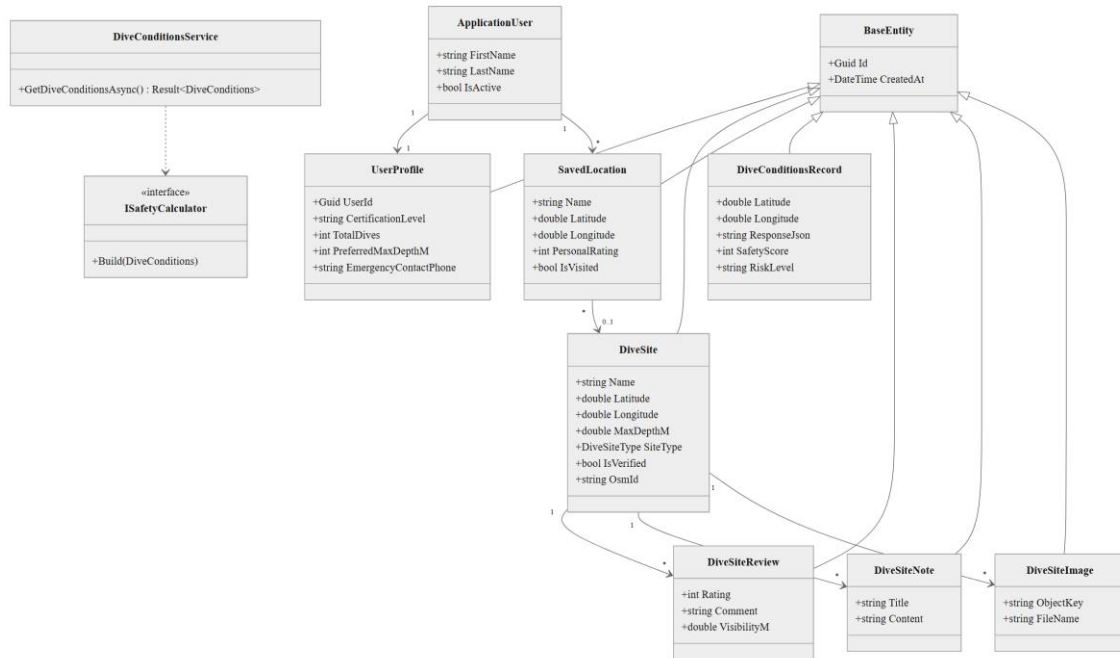
- усі залежності спрямовані до Core;
- BLL використовує DAL та Integrations;
- контролери — тонкий шар над сервісами;
- калькулятори безстанові (Singleton у DI).



UML-моделювання системи

Розроблено повний набір UML-діаграм:

- use-case — сценарії чотирьох ролей;
- класів — доменні сутності та сервіси;
- послідовності та діяльності — поведінка;
- компонентів і розгортання — структура.

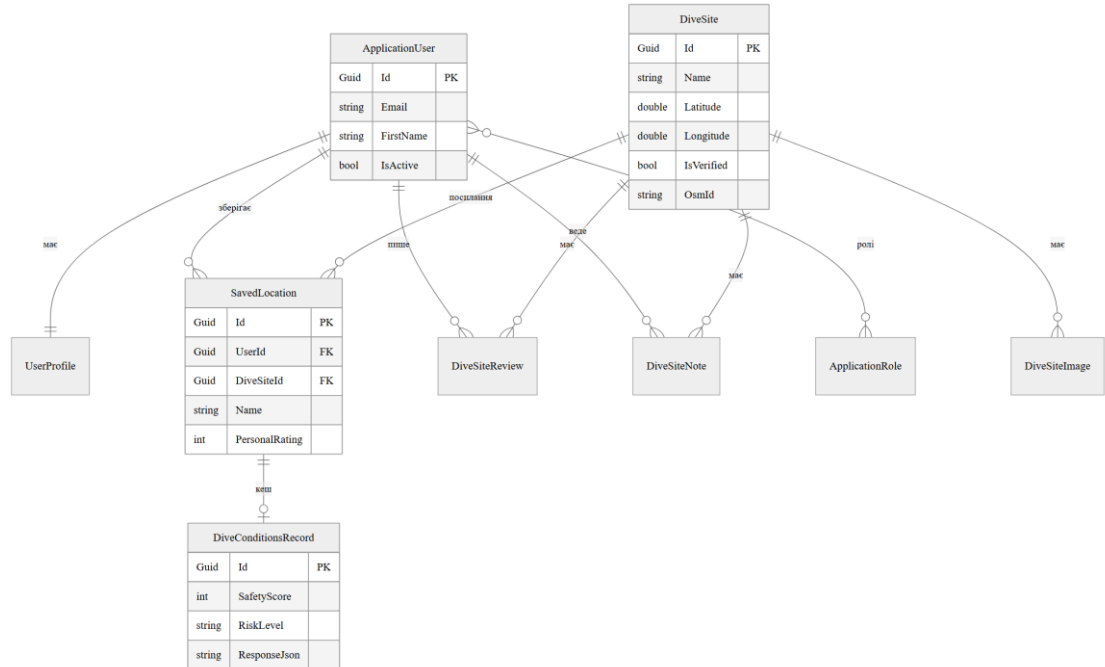


Модель даних та схема БД

Три групи сутностей:

- користувачі, профілі дайверів, ролі;
- дайв-сайти, відгуки, нотатки, зображення;
- збережені локації та кеш умов занурення.

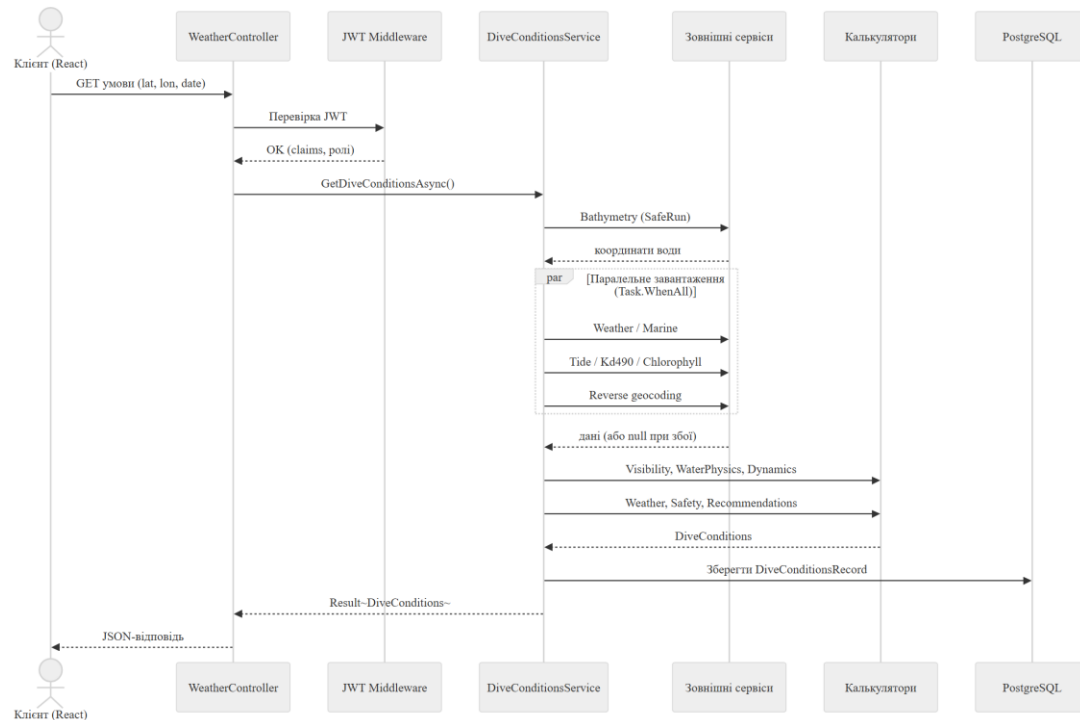
PostgreSQL, EF Core, Repository-патерн, індекси для геопошуку.



Конвеєр оцінки умов

Детермінований конвеєр з 1 етапів:

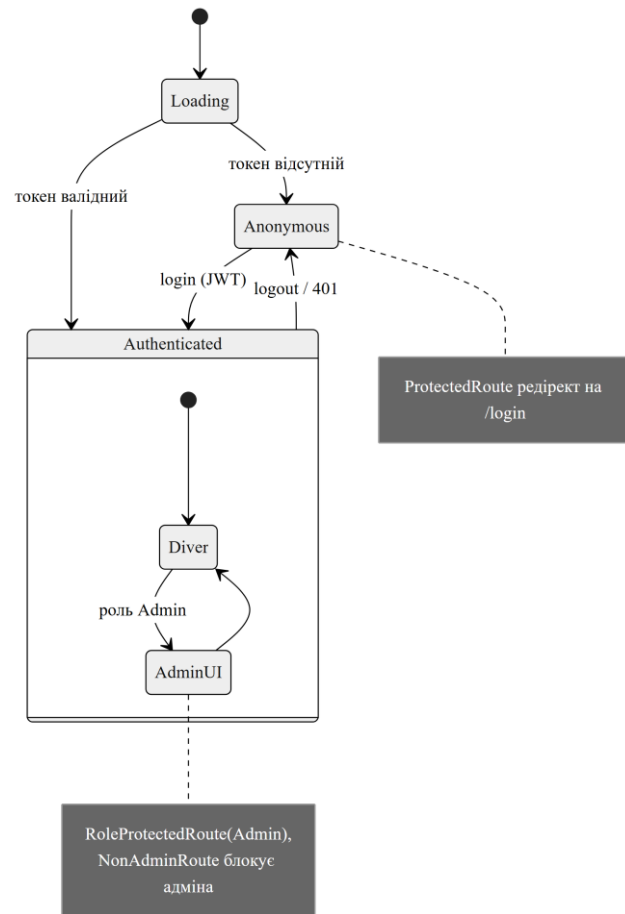
- батиметричний аналіз точки
- паралельне завантаження даних (Task.WhenAll);
- калькулятори: видимість, термоклин, динаміка;
- оцінка безпеки 0–100 та рекомендації.



Клієнтська частина (React 19)

Архітектура SPA:

- функціональні компоненти та хуки;
- глобальний стан через React Context;
- три типи захищених маршрутів;
- локалізація (i18next), Tailwind CSS.



Процес розробки

Етапи розробки

- проектування архітектури → серверна частина → клієнт →
- інтеграції й тестування → контейнеризація й розгортання;

Мови, фреймворки та обсяг

- C# 13 / ASP.NET Core (сервер); TypeScript / React 19 (клієнт);
- 6 проєктів, ~250 файлів, близько 15 000 рядків коду.

Приклад реалізації: SafeRun

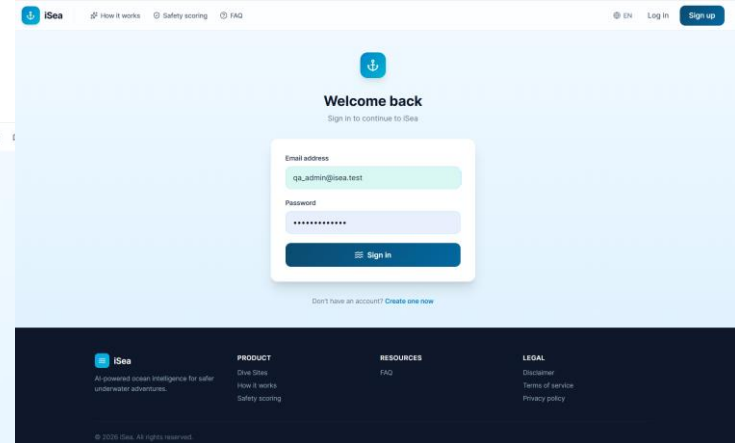
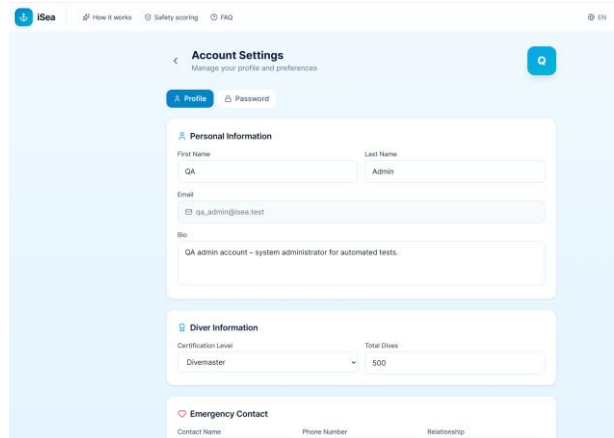
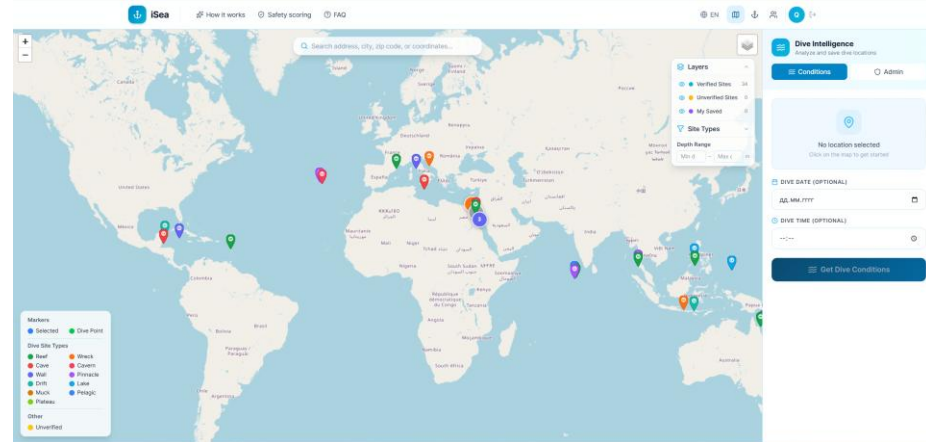
Кожен виклик зовнішнього сервісу обгорнуто у відмовостійку функцію SafeRun:

```
private async Task<T> SafeRun<T>(Func<Task<T>> fn, string integrationName)
{
    try
    {
        return await fn();
    }
    catch (Exception ex)
    {
        _logger.LogWarning(ex,
            message: "Upstream integration {Integration} failed; continuing with degraded data",
            integrationName);
        return default!;
    }
}
```

Інтерфейс користувача

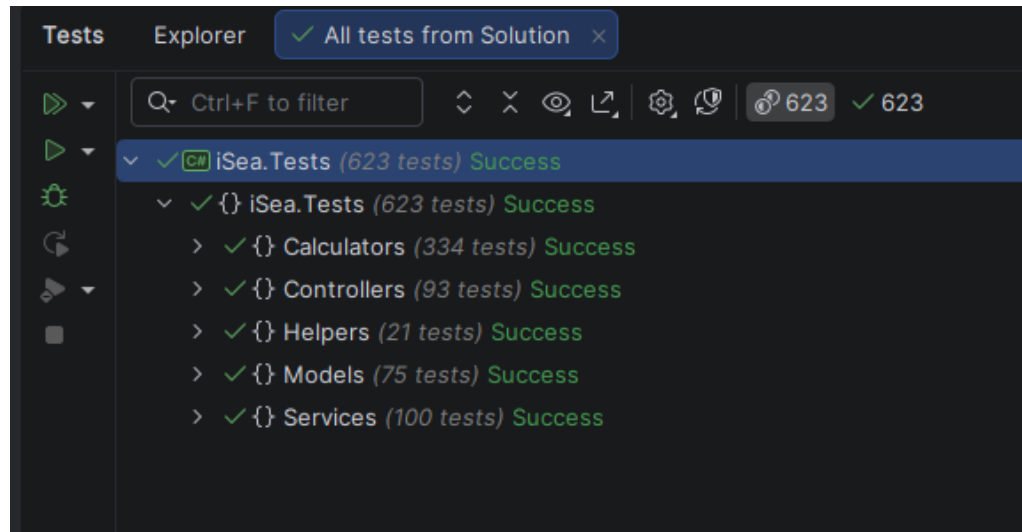
Основні екрани застосунку:

- головна сторінка та опис можливостей платформи;
- інтерактивна мапа дайв-сайтів з кластеризацією маркерів;
- модальне вікно з результатами оцінки умов занурення;
- особистий кабінет, збережені локації та відгуки.



Тестування

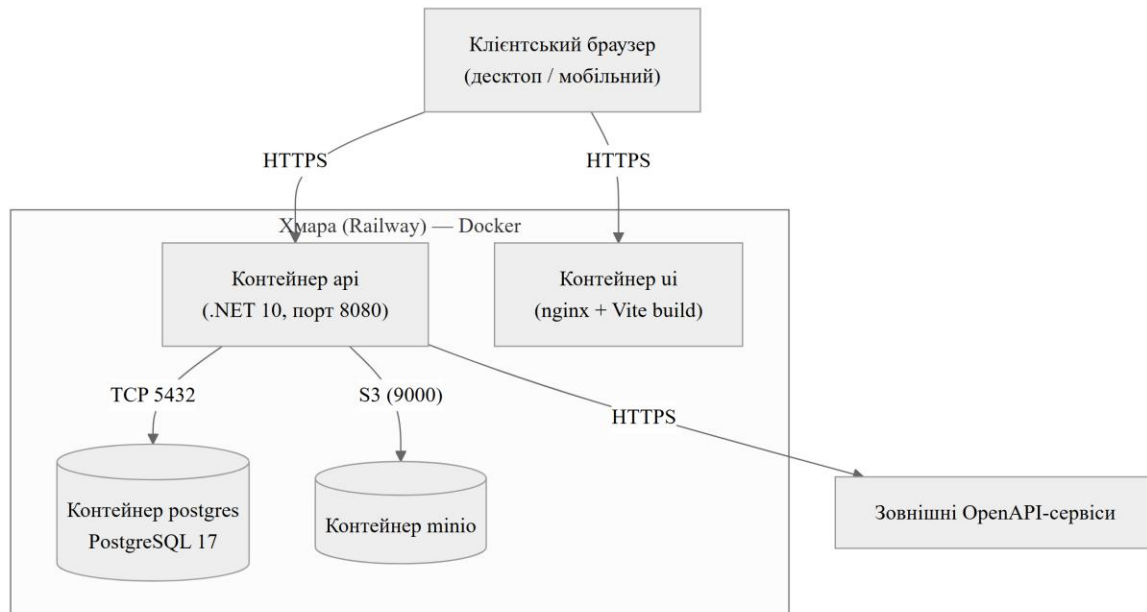
- понад 579 модульних тестів на xUnit;
- покриття ключових калькуляторів і сервісів понад 80%;
- сценарії: типові, граничні, помилкові, з null-значеннями;
- інтеграційні тести клієнтів зовнішніх сервісів;
- клієнтські тести критичних компонентів (Vitest);
- **усі тести проходять успішно.**



Розгортання

Контейнеризація та хмара:

- Docker Compose: api, ui, postgres, minio;
- повний стек запускається однією командою;
- хмарне розгортання на Railway;
- автоматичні EF-міграції та health-checks.



Підсумки

- реалізовано повноцінний повностековий веб-застосунок iSea;
- усі поставлені задачі виконано в повному обсязі;
- єдина інтегральна ШІ-оцінка умов — перевага над аналогами;
- відмовостійкість і розгортання однією командою;

Перспективи розвитку

- мобільні застосунки, push-сповіщення, ML-уточнення оцінок.

